イベントストリーム

DJ が配信している間の出来事を Server-Sent Events(SSE)でリアルタイムに受け取ります。ひとつの接続でチャット・入場・ハート・ギフトをまとめて受け取り、同意された権限に該当する イベントだけが流れます。

メソッド・パス	`GET /v1/live/events`
必要な権限	`events.chat`・`events.presence`・`events.like`・`events.donation` の **いずれかひとつ以上**
成功レスポンス	`200`(`text/event-stream`)
配信中でないとき	`404`(`OAPI_MNGR_0301`)

```
export SPOON_BASE_URL="https://jp-openapi.spooncast.net"
export SPOON_ACCESS_TOKEN="{発行された access token}"
```

韓国リージョンなら `https://kr-openapi.spooncast.net` です — リージョン別アドレス。

何が起こるのか

```
トークン発行 ─▶ ストリーム接続 ─▶ 配信に入場 ─▶ イベント受信
                    │                              │
                    └ 404: 配信中でない              └ end: ストリーム終了
                      バックオフして再試行             接続に戻る
```

トークンが配信を決めます

パスにもクエリにも DJ を書く場所がありません。トークン 1 つが同意した DJ 1 人なので、どの トークンを使うかがそのままどの配信を扱うかになります。ボットが他人の配信を指定することはできません。

接続するとボットがその配信に入場します

単なる購読ではありません。接続した瞬間、ボットアカウントが **リスナーとして部屋に入ります。** 3 つのことがついてきます。

- **ボット自身はどこにも出ません。** イベントストリームだけでなく リスナー一覧・ファンランキング にも出ません。抽選ボットが自分を引くことはありません。
- **DJ の制御をそのまま受けます。** 強制退出されるとイベントが止まり、チャット禁止・凍結なら 発言が塞がれます。
- **受け取れる範囲はボットの部屋での権限に従います。** マネージャーに指定されればマネージャー 等級のメッセージまで届きます。

接続が切れても入場は維持されます

ネットワークやデプロイでストリームが切れてもボットは部屋に残ります。再接続すればそのまま続きます。**再接続だけでは入場メッセージは再び表示されません。** サーバーは接続時にボットを部屋に入れますが、すでに入っていればそのまま続くためです。

ボットが部屋から**抜けた後**に再び接続する(またはチャットを送る)と、新しい入場として扱われ、入場メッセージがもう一度表示されます。ボットが部屋から抜けるのは 2 つの場合です。

- DJ がボットを強制退出させたとき
- 入場記録が失効したとき — 最初に入場してから 5 時間で失効します。再入場しても延長されません

同じ DJ への接続はひとつだけ維持します

ひとつのアプリが同じ DJ の配信に開いておけるストリームはひとつです。同じ DJ が何度か同意してトークンが複数あっても同じです。

新しい接続が入ると、**前の接続は 15 秒以内に閉じます。** 前の接続は閉じる前に `end` フレームで `DUPLICATE_CONNECTION` を受け取ります。この理由を受け取ったら **接続し直さないでください。** 接続し直すと今度は相手側が閉じ、ふたつの接続が交互に切り合います。

ボットサーバーを複数立てる場合は、ひとりの DJ をひとつのサーバーだけが担当するように分けます。

終わるときは `end` で理由を知らせて閉じます

`end` フレームの後には何も届きません。**理由ごとにボットのすることが違います** — 配信が 終わったのか、トークンが失効したのか、サーバーがこの接続だけを閉じたのか、同じアプリの別の接続が引き継いだのかを `reason` が伝えます。

接続するとリスナー向け告知がボット名で送られます

接続している間、配信に **ボットアカウント名でリスナーへの告知** が投稿されます。オフにはできず、ボットのコードですることもありません — いつ、誰に送られるかは[リスナーへの告知]にあります。

リクエスト

```
GET /v1/live/events
```

ヘッダー

名前	必須	説明
`Authorization`	✔	`Bearer {access_token}` の形式です。
`Accept`	✔	`text/event-stream` です。

パスパラメータ・クエリパラメータ・リクエストボディは **ありません。どの DJ の配信かはトークンが決めます。**

必要な権限: `events.chat`・`events.presence`・`events.like`・`events.donation` の **いずれかひとつでも** あれば接続が開きます。複数あればそれらのイベントが同じ接続で届きます。同意されていないイベントは除外されるため、接続は正常なのにそのイベントだけ届きません。ひとつもなければ `403` です。

リクエスト例

```
curl -N \
  -H "Authorization: Bearer $SPOON_ACCESS_TOKEN" \
  -H "Accept: text/event-stream" \
  "$SPOON_BASE_URL/v1/live/events"
```

レスポンス

`200` でストリームが開き、その後 SSE のフレームが続きます。

```
:heartbeat
```

```
id:1788328046114-0
event:chat
data:{"user":{"id":"Ab1Cd2Ef3Gh4Ij5Kl6Mn7O","nickname":"スプーンリスナー"},"isDj":false,"message":"こんにちは","sentTime":"2026-09-02T05:47:26.114Z"}

id:1788328052169-1
event:presence
data:{"user":{"id":"Pq8Rs9Tu0Vw1Xy2Za3Bc4D","nickname":"ラジオファン"},"type":"JOIN","fanRank":1,"isManager":false,"favoriteTemperature":36.5,"time
09-02T05:47:32.169Z"}

id:1788328061044-0
event:like
data:{"user":{"id":"Pq8Rs9Tu0Vw1Xy2Za3Bc4D","nickname":"ラジオファン"},"type":"PAID","totalAmount":30,"amount":10,"extraAmount":5,"combo":2,"time":
09-02T05:47:41.044Z"}

id:1788328070512-0
event:donation
data:{"user":{"id":"Ab1Cd2Ef3Gh4Ij5Kl6Mn7O","nickname":"スプーンリスナー"},"amount":300,"message":"応援しています","time":"2026-09-02T05:47:50.512Z"}

event:end
data:{"reason":"LIVE_ENDED"}
```

イベント	必要な権限	いつ届くか	ペイロード
`chat`	`events.chat`	リスナーまたは DJ がチャットを打ったとき	chat
`presence`	`events.presence`	リスナーが配信に入場したとき	presence
`like`	`events.like`	リスナーがハートを送ったとき	like
`donation`	`events.donation`	リスナーがギフトを送ったとき	donation
`end`	なし	ストリームが終わるとき。このフレームの後には何も届きません	end

`id:` は `{epochMillis}-{offset}` の形式です。現在は再開に使われませんが、後で `Last-Event-ID` による再開を開くときに契約が壊れないよう、最初から付いています。

ハートビート

```
:heartbeat
```

`:` で始まる SSE の comment 行が既定で 15 秒ごとに届きます。チャットがなくても接続を 維持することだけが目的です。

標準の `EventSource` はこの行を自動的に無視するので、気にする必要はありません。自分で パーサーを書くときだけ、`data:` がないフレームとして除外してください。

イベントを受け取る

以下のサンプルを保存して実行すると、4 つのイベントがすべてコンソールに表示されます。各 イベント節のコードはこの `handlers` に入れます。

```ts
// stream.ts — node --experimental-strip-types stream.ts

interface EventUser {
  id: string;
  nickname: string | null;
}

interface ChatEvent {
  user: EventUser;
  isDj: boolean;
  message: string;
  sentTime: string;
}

interface PresenceEvent {
  user: EventUser;
  type: 'JOIN';
  fanRank: number | null;
  isManager: boolean;
  favoriteTemperature: number | null;
  time: string;
}

interface LikeEvent {
  user: EventUser;
  type: 'FREE' | 'PAID';
  totalAmount: number;
```

```
  amount: number;
  extraAmount: number;
  combo: number | null;
  time: string;
}

interface DonationEvent {
  user: EventUser;
  amount: number;
  message: string | null;
  time: string;
}

interface StreamEndEvent {
  reason: 'LIVE_ENDED' | 'TOKEN_EXPIRED' | 'RECONNECT' |
'DUPLICATE_CONNECTION';
}

const res = await fetch(`${process.env.SPOON_BASE_URL}/v1/live/events`, {
  headers: {
    Authorization: `Bearer ${process.env.SPOON_ACCESS_TOKEN}`,
    Accept: 'text/event-stream',
  },
});
if (!res.ok || !res.body) throw new Error(`ストリームを開けませんでした:
${res.status}`);

const handlers: Record<string, (d: any) => void> = {
  chat: (d: ChatEvent) => console.log(`${d.user.nickname ?? '匿名'}:
${d.message}`),
  presence: (d: PresenceEvent) => console.log(`${d.user.nickname ?? '匿名'} 入
場`),
  like: (d: LikeEvent) => console.log(`ハート ${d.totalAmount}個 (${d.type})`),
  donation: (d: DonationEvent) => console.log(`ギフト ${d.amount} スプーン`),
  end: (d: StreamEndEvent) => console.log(`ストリーム終了: ${d.reason}`),
};

const reader = res.body.pipeThrough(new TextDecoderStream()).getReader();
let buffer = '';

for (;;) {
  const { value, done } = await reader.read();
  if (done) break;
  buffer += value;

  const frames = buffer.split('\n\n');
  buffer = frames.pop() ?? '';
```

```
  for (const frame of frames) {
    const lines = frame.split('\n');
    const event = lines.find((l) => l.startsWith('event:'))?.slice(6);
    const data = lines.find((l) => l.startsWith('data:'))?.slice(5);
    if (!event || !data) continue;   // `:heartbeat` には data がありません
    handlers[event]?.(JSON.parse(data));
  }
}
```

```
node --experimental-strip-types stream.ts
```

```
node --experimental-strip-types stream.ts
```

ブラウザの `EventSource` では接続できません。トークンを `Authorization` ヘッダーで送りますが `EventSource` はヘッダーを付けられません。サーバーで上のコードを動かし、結果だけをブラウザへ 中継します。

イベントペイロード

すべてのイベントのユーザー情報は同じ形です。

フィールド	型	null	説明
`user.id`	string	いいえ	その人を指す値です。暗号化した値です。
`user.nickname`	string	**はい**	ニックネームです。空文字列ではなく `null` が来ます。

> **`id` で人を識別します**
>
> `id` はその人の一意な識別子です。同じ人なら常に同じ値なので、出席・累計・ランキングのキーとして 使います。`nickname` は変わり、重複するので識別に使わないでください。
>
> 値の範囲は自分のボットとその DJ の組み合わせです。DJ が連携を解除して再度同意しても維持されます。

この `id` は リスナー一覧・ファンランキング の `id` と **同じ値** です。

`chat` — チャット

リスナーのチャットと DJ のクイックメッセージをリアルタイムに受け取ります。**必要な権限**: `events.chat`

リスナーの発言:

```
{
  "user": { "id": "Ab1Cd2Ef3Gh4Ij5Kl6Mn7O", "nickname": "スプーンリスナー" },
  "isDj": false,
  "message": "こんにちは",
  "sentTime": "2026-09-02T05:47:26.114Z"
}
```

DJ の発言 — `isDj` が `true` です:

```
{
  "user": { "id": "Uv4Wx5Yz6Ab7Cd8Ef9Gh0I", "nickname": "スプーンDJ" },
  "isDj": true,
  "message": "お便り送ってね！",
  "sentTime": "2026-09-02T05:47:31.802Z"
}
```

フィールド	型	null	説明
`user.id`	string	いいえ	発言した人です。暗号化した値です。
`user.nickname`	string	**はい**	ニックネームです。
`isDj`	boolean	いいえ	この発言が DJ 本人のものかを示します。`user` の中ではなく最上位にあります。
`message`	string	いいえ	チャットの原文です。
`sentTime`	string	いいえ	送信時刻です。ISO-8601 UTC(`Z`)です。

```
// stream.ts — handlers に `chat: onChat` として入れます
```

```
const counts = new Map();

const onChat = (d: ChatEvent) => {
  if (d.isDj) return;                       // DJ は集計から外す
  const n = (counts.get(d.user.id) ?? 0) + 1;
  counts.set(d.user.id, n);                 // キーは nickname ではなく id
  console.log(`${d.user.nickname ?? '匿名'}: ${d.message} (${n}回目)`);
};
```

`isDj` **を見ないと DJ が自分の配信でポイントを稼ぎます。** DJ もチャットを打ち、クイック メッセージを送ります。出席やランキングの集計から外さないと、DJ が自分のランキング 1 位に なります。暗号化された `id` からは誰が DJ か分からず、ニックネームの比較も答えではないので、このフィールドで分けます。

`isDj` は `chat` **にだけあります。** `user` の中ではなくイベントの最上位です。他のイベントには このフィールド自体がありません — 原本が与えないので、常に `false` の値を作って送りません。

`nickname` が `null` **のことがあります。** 空文字列ではなく `null` です。`d.user.nickname ?? '匿名'` のように受けます。

`message` **が JSON のように見えても文字列です。** リスナーは `{"a":1}` を打てます。パースしないでください。

チャットを禁止されたリスナーは発言そのものができません。 そのため、そのリスナーの `chat` イベントはそもそも作られません — こちらで除外しているのではなく、作られるものが ありません。禁止は **その配信の中でのみ** 有効で、次の配信には引き継がれず、そのときは 同じ人のチャットがまた流れます。

ギフトメッセージ・運営のお知らせ・他のシステム発言は届きません。ボット自身の発言も戻りません。

`presence` — 入場

リスナーが配信に入場すると受け取ります。**必要な権限**: `events.presence`

```
{
  "user": { "id": "Pq8Rs9Tu0Vw1Xy2Za3Bc4D", "nickname": "ラジオファン" },
  "type": "JOIN",
  "fanRank": 1,
  "isManager": false,
  "favoriteTemperature": 36.5,
  "time": "2026-09-02T05:47:32.169Z"
}
```

フィールド	型	null	説明
`user.id`	string	いいえ	入場した人です。暗号化した値です。
`user.nickname`	string	**はい**	ニックネームです。
`type`	string	いいえ	現在は `JOIN` のみ届きます。
`fanRank`	number	**はい**	この配信のギフトランキング順位です。ギフト履歴がなければ `null` です。
`isManager`	boolean	いいえ	DJ がこのリスナーをマネージャーに指定しているかを示します。
`favoriteTemperature`	number	**はい**	愛聴温度です。配信が変わっても引き継がれる累積の関係指標なので、常連を見分けるのに使います。
`time`	string	いいえ	入場時刻です。ISO-8601 UTC(`Z`)です。

入場の時点でランキング・マネージャー・愛聴温度が一緒に届きます。「1 位のファンが入場！」や「常連へのあいさつ」を作るのに ファンランキングを取得 を別途呼ぶ必要はありません。

```
// stream.ts — handlers に `presence: onPresence` として入れます
const greeted = new Set();

const onPresence = (d: PresenceEvent) => {
  if (greeted.has(d.user.id)) return;     // 再入場に二度あいさつしない
  greeted.add(d.user.id);

  const name = d.user.nickname ?? '匿名';
  if (d.fanRank === 1) console.log(`${name}さん、1 位のファンが入場！`);
  else if (d.isManager) console.log(`マネージャーの ${name}さん、いらっしゃい！
`);
  else if (d.favoriteTemperature !== null && d.favoriteTemperature >= 36.5)
    console.log(`常連の ${name}さん、お久しぶりです！`);
  else console.log(`${name}さん、いらっしゃい！`);
};
```